HSL-530

Data Analytics for Business & Society

Phase II Project Report

RetailHero Uplift Modeling: CCP vs CCU Evaluation

Submitted by

Anand Deep (21322003)
Aniket (21322005)
Madhav Mukund (21322018)
Param Mahabare (21322021)

29 April 2026

Abstract

This report presents the Phase II implementation and evaluation of a value-aware uplift-modeling pipeline for the RetailHero / X5 retail intervention dataset. Phase I established that campaign assignment was effectively randomized, that the average treatment effect was positive but modest, and that treatment response varied materially across customer segments. Phase II builds on those findings by comparing a conventional customer purchase-propensity benchmark (CCP) with three customer churn uplift (CCU) estimators under a common experimental framework. Features are reconstructed directly from the raw source files, models are trained on a stratified development split, and performance is assessed on a held-out validation sample using Qini, liftup, predictive maximum profit, and maximum profit uplift. The empirical results indicate that the uplift random forest provides the strongest incremental-targeting performance, whereas the CCP benchmark is most appropriately interpreted as a non-causal propensity baseline.

Contents

1 Introduction and Objective **1**

2 Data and Verified Phase I Findings **1**
2.1 Dataset . . . 1
2.2 Phase I Findings Revalidated from Raw Data . . . 1
2.3 Implications for Phase II Design . . . 2

3 Phase II Methodology **2**
3.1 Design Principles . . . 2
3.2 Feature Engineering . . . 2
3.3 Data Split . . . 3
3.4 Models . . . 3
3.4.1 Model A: CCP baseline . . . 3
3.4.2 Model B1: Two-model logistic uplift . . . 3
3.4.3 Model B2: Lo-style interaction logistic uplift . . . 3
3.4.4 Model B3: Uplift random forest . . . 4
3.5 Evaluation Framework . . . 4

4 Implementation Summary **4**
4.1 Code Organization . . . 4
4.2 Implementation Integrity Checks . . . 4

5 Results **5**
5.1 Selected Hyperparameters . . . 5
5.2 Validation Summary . . . 5
5.3 Interpretation . . . 5
5.4 Figures . . . 6

6 Limitations **8**

7 Conclusion **8**

A Selected Code Excerpts and Explanation **8**
A.1 Client Cleaning and Redemption Feature Construction . . . 9
A.2 Product Behavior Aggregation . . . 9
A.3 Deterministic Missing-Value Handling . . . 11
A.4 Linear Uplift Scoring Logic . . . 12
A.5 Benchmark and Non-Linear Uplift Wrappers . . . 13
A.6 End-to-End Orchestration . . . 14

B Reproduction Commands **16**

1 Introduction and Objective

The objective of Phase II was to construct a reproducible and methodologically coherent comparison between:

- **Model A: CCP baseline**, a standard purchase-propensity model trained only on the control group.
- **Model B: CCU models**, uplift estimators that rank customers by expected incremental response to treatment.

The central research question addressed in this phase is as follows:

> Can the intervention be targeted more efficiently by ranking customers on expected incremental value rather than raw purchase likelihood?

This distinction is important because a customer with a high unconditional probability of purchase may still be an inefficient target for campaign expenditure if that customer would have purchased in the absence of treatment. The practical value of uplift modeling therefore depends on its ability to alter the targeting order in a measurable and economically meaningful way.

2 Data and Verified Phase I Findings

2.1 Dataset

The project uses the RetailHero uplift dataset with the following raw inputs:

- `clients.csv`
- `products.csv`
- `purchases.csv`
- `uplift_train.csv`
- `uplift_test.csv`
- `uplift_sample_submission.csv`

The purchases file is the dominant data asset and contains approximately 4.4 GB of transaction history. Because of its size, the implementation uses DuckDB-backed SQL aggregation rather than loading the entire file into a single in-memory pandas frame.

2.2 Phase I Findings Revalidated from Raw Data

Phase I findings were rechecked directly from raw data via the verification script rather than relying on notebook narrative alone. The key findings are summarized in Table 1.

Table 1: Verified Phase I findings used to motivate Phase II

Finding	**Verified value**
Treatment / control ratio	0.999230 (99,981 treated / 100,058 control)
Baseline uplift	3.323 percentage points
Average uplift in low-frequency deciles 0–3	4.452 percentage points
Average uplift in high-frequency deciles 6–9	1.755 percentage points
Sure Things share	30.176% of training clients
Age anomalies	min = -7,491, max = 1,901, flagged = 1,145
Distinct non-null `level_1` categories	3
Test-set clients with purchase history	200,123 / 200,123

2.3 Implications for Phase II Design

These findings motivate the Phase II design in three ways:

1. The near-perfect treatment/control balance means uplift estimates can be interpreted causally without heavy post-hoc adjustment.
2. The modest average treatment effect means a single global response model is likely to be wasteful because the mean hides heterogeneity.
3. The concentration of uplift among lower-frequency customers means targeting by purchase probability alone is not aligned with incremental value.

3 Phase II Methodology

3.1 Design Principles

Phase II was built around four design principles:

1. **Raw-data-first reproducibility**: all Phase II features are rebuilt from `data/*.csv` rather than notebook-only exports.
2. **Comparable feature space**: all models consume the same deterministic numeric feature matrix.
3. **Value-aware evaluation**: uplift models are not judged only by response prediction, but by incremental targeting value.
4. **Transparent implementation**: the pipeline is organized into feature, model, metric, and orchestration modules rather than one monolithic script.

3.2 Feature Engineering

The final Phase II matrix contains 44 numeric features with zero missing values after deterministic filling and explicit missingness indicators. The feature groups are:

- **Demographics**: `age`, `age_clean`, `age_flagged`, gender indicators, `gender_known`
- **Redemption behavior**: redeemed flag, time-to-redeem, card tenure, and redemption recency

- **Purchase behavior**: spend, transactions, trips, items, basket metrics, stores, products, points, recency, tenure
- **Product behavior**: alcohol spend share, own-trademark spend share, netto aggregates, spend-share across the three `level_1` groups, and brand concentration (HHI)

This feature design follows directly from the empirical lessons of Phase I. Because the hashed category hierarchy is not semantically interpretable, the implementation does not impose artificial business meaning on those fields. Instead, product metadata is converted into client-level behavioral aggregates that preserve signal while remaining methodologically honest.

3.3 Data Split

The Phase II pipeline uses:

- an outer holdout split of 20% of `uplift_train` for validation,
- stratification on the joint label `(treatment_flg, target)`,
- a fixed random seed of 123,
- a smaller inner tuning split inside the training fold only.

This split design preserves an untouched validation sample while keeping the modeling pipeline sufficiently transparent to inspect, reproduce, and extend.

3.4 Models

3.4.1 Model A: CCP baseline

The CCP baseline is an `XGBClassifier` trained on control-group rows only. It predicts purchase propensity rather than treatment effect. This specification provides a strong conventional benchmark, avoids learning treatment-induced behavior when estimating organic demand, and reflects the type of model that might plausibly be deployed if uplift were ignored.

3.4.2 Model B1: Two-model logistic uplift

Two separate regularized logistic models are trained, one on treated customers and one on control customers. The uplift score is:

$$\hat{u}(x) = \hat{P}(Y = 1 \mid T = 1, X = x) - \hat{P}(Y = 1 \mid T = 0, X = x)$$

This specification is the most direct implementation of the roadmap formula and serves as a transparent linear uplift baseline.

3.4.3 Model B2: Lo-style interaction logistic uplift

A single logistic model is trained on all rows with:

- the treatment indicator as a feature,
- first-order interactions between treatment and all covariates.

The uplift score is obtained by scoring each customer twice, once under treatment and once under control, then taking the difference.

This specification provides a structured single-model alternative to the two-model approach and is useful for testing whether treatment interactions can be captured parsimoniously within one regularized estimator.

3.4.4 Model B3: Uplift random forest

The non-linear uplift model is implemented using `causalml`'s `UpliftRandomForestClassifier` with:

- `KL` split evaluation,
- string-encoded treatment labels required by the package API,
- light tuning over tree count, feature count, depth, leaf size, and regularization.

This model is the most expressive estimator in the comparison and is capable of representing non-linear treatment heterogeneity that the linear alternatives cannot capture.

3.5 Evaluation Framework

Phase II evaluates the models with four complementary views:

- **Qini curves**: how well the model ranks incremental responders
- **Liftup curves**: uplift-specific analogues of lift behavior
- **Predictive maximum profit (MP)**: used for the CCP benchmark
- **Maximum profit uplift (MPU)**: used for incremental targeting

The default economics are:

- average basket value = empirical average trip value = 4425.003 RUB
- SMS cost = 1 RUB

Sensitivity analysis then varies both assumptions.

4 Implementation Summary

4.1 Code Organization

The Phase II codebase is modular:

- `phase2_features.py`: raw-data feature construction and cache writing
- `phase2_models.py`: model wrappers, tuning, and scoring
- `phase2_metrics.py`: ranking, Qini, liftup, MP, and MPU calculations
- `phase2_pipeline.py`: full orchestration and artifact generation
- `verify_phase2_outputs.py`: artifact and consistency checks

4.2 Implementation Integrity Checks

The completed pipeline passed all current verification checks:

- 200,039 Phase II train rows and 200,123 Phase II test rows
- 44 model features
- zero missing feature values in both train and test matrices

- valid category-share algebra with maximum absolute error 1.343×10^{-14}
- finite validation and test scores for all four models
- valid target-share bounds for Qini, liftup, and sensitivity outputs

These checks are important because a submission-ready analysis should demonstrate not only competitive model performance, but also internal consistency of the underlying data and artifact pipeline.

5 Results

5.1 Selected Hyperparameters

The final selected hyperparameters from the current full run are shown in Table 2.

Table 2: Selected model hyperparameters

Model	Selected parameters
Model A - CCP baseline	`n_estimators=250`, `max_depth=3`, `learning_rate=0.05`, `subsample=0.85`, `colsample_bytree=0.85`
Model B1 - Two-model logistic	`C=1.0`
Model B2 - Lo interaction logistic	`C=10.0`
Model B3 - Uplift random forest	`n_estimators=25`, `max_features=12`, `max_depth=5`, `min_samples_leaf=100`, `min_samples_treatment=50`, `n_reg=10`

5.2 Validation Summary

The full validation summary is shown in Table 3. The validation holdout contains 40,008 rows.

Table 3: Phase II validation summary

Model	Qini AUC	Top-decile liftup	MPU	Optimal target share
Model B3 - CCU Uplift Random Forest	0.004759	3.637315	158.307694	0.7475
Model B1 - CCU Two-Model Logistic	0.003297	3.239985	146.083912	1.0000
Model B2 - CCU Lo Interaction Logistic	0.003245	3.231146	146.083912	1.0000
Model A - CCP Baseline	-0.002239	0.016284	146.083912	1.0000

For completeness, the CCP baseline also achieved a predictive maximum profit of 2742.063982 under the default economics. That figure is intentionally reported separately from MPU because it is not the same business question. Predictive MP rewards finding likely buyers. MPU rewards finding customers whose behavior is changed by treatment. Those are related but not interchangeable objectives.

5.3 Interpretation

Three observations are particularly important:

1. The uplift random forest is the strongest CCU model on both Qini and MPU.
2. The logistic uplift models show positive uplift discrimination and materially outperform the CCP baseline on top-decile liftup.
3. The CCP baseline is excellent at ranking likely buyers, but poor at isolating persuadable buyers near the top of the list.

This pattern is consistent with the evidence reported in Phase I. High-frequency customers often behave as "sure things" and can therefore dominate a purchase-propensity ranking while contributing relatively little incremental value. The uplift models correct for this by ranking customers according to estimated treatment effect rather than unconditional purchase likelihood.

5.4 Figures

Figure 1 shows the Qini curves. The uplift random forest dominates the uplift-specific ranking metric, while the CCP baseline is negative relative to random on this criterion.

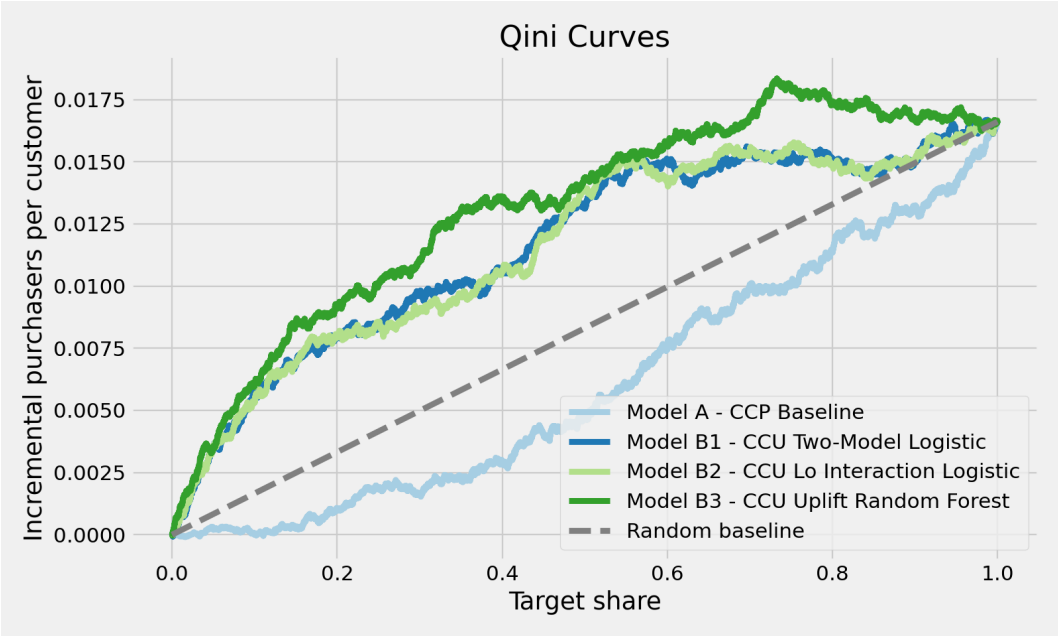

Figure 1: Qini curves on the Phase II validation holdout.

Figure 2 shows the liftup curves. The CCP baseline has very high top-decile response lift but almost no top-decile liftup, which is precisely the failure mode Phase I warned about.

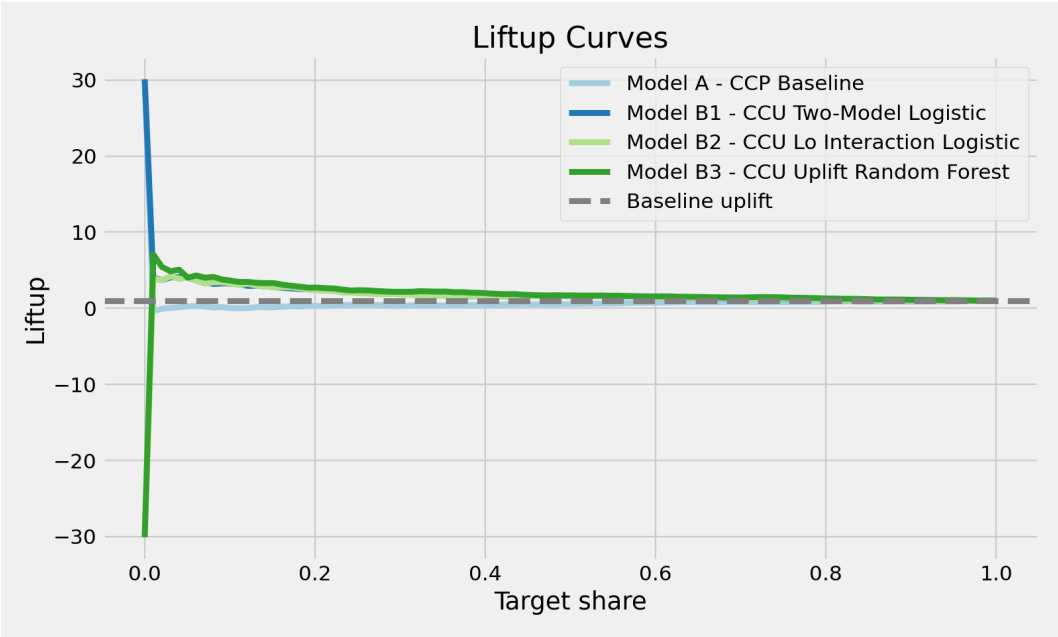

Figure 2: Liftup curves on the Phase II validation holdout.

Figure 3 compares incremental profit curves and the CCP predictive MP curve. The uplift random forest is the only model that clearly pushes the incremental-profit frontier upward.

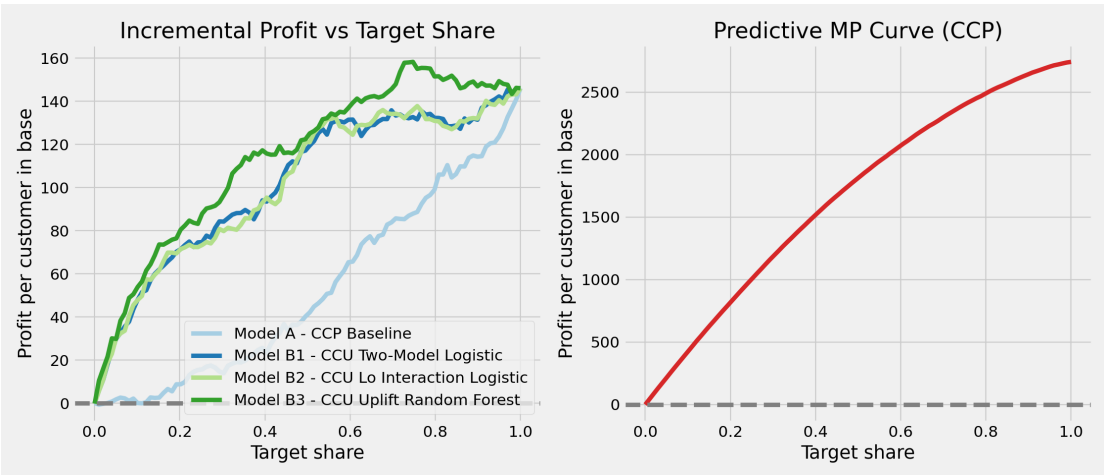

Figure 3: Incremental profit and CCP predictive MP curves.

Figure 4 shows that the relative ordering is not a single-point artifact of the chosen economics; the uplift random forest remains competitive under the current sensitivity sweep.

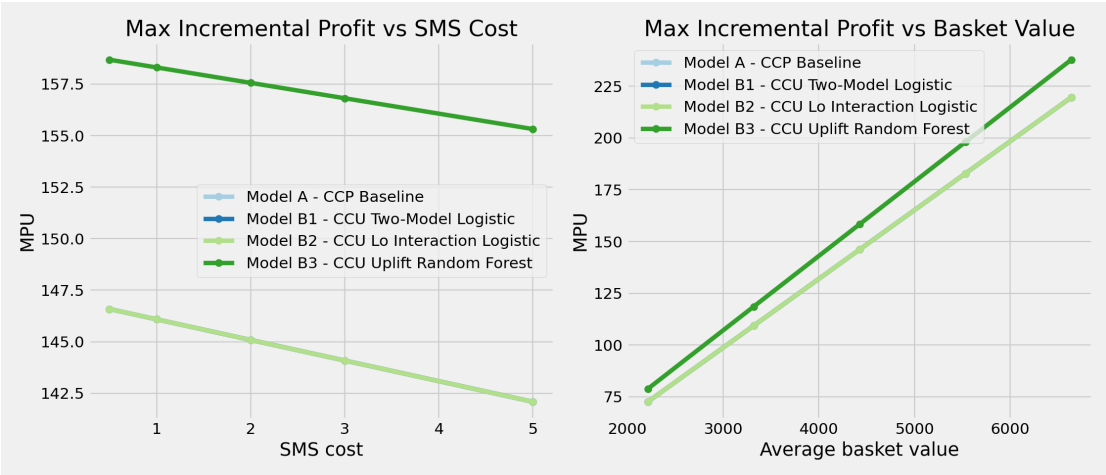

Figure 4: Sensitivity of maximum incremental profit to basket value and SMS cost assumptions.

6 Limitations

Several limitations remain:

- The hashed product hierarchy limits semantic interpretation.
- The current profit analysis depends on default economic assumptions, though sensitivity analysis reduces that risk.
- The uplift random forest uses a heavier third-party package and is slower than the logistic baselines.
- The implementation is an experimental analysis pipeline, not a production scoring service.

These limitations are acceptable at the present stage because the purpose of Phase II was comparative methodology and defensible empirical analysis rather than production deployment.

7 Conclusion

Phase II successfully translated the Phase I findings into a reproducible uplift-modeling comparison. The resulting pipeline demonstrates that:

- a conventional CCP purchase-propensity ranking is not sufficient for incremental targeting,
- uplift-specific models are empirically justified on this dataset,
- the uplift random forest is currently the strongest model under the implemented evaluation framework.

Taken together, the results support the central claim of the project: customer targeting based on expected incremental response is more defensible than targeting based solely on unconditional purchase probability.

A Selected Code Excerpts and Explanation

Only selected excerpts are reproduced below. The purpose of this appendix is to document the decisive implementation logic without reproducing complete source files. Full source code remains available in the project repository; the report includes only those portions that directly support the methodological claims made in the main body.

A.1 Client Cleaning and Redemption Feature Construction

Listing 1 shows the client-level preprocessing block. This excerpt is important because it demonstrates how Phase I data-quality findings were translated into modeling inputs rather than merely documented descriptively.

```python
def _read_clients(clients_path: str | Path, campaign_date: pd.Timestamp) -> pd.DataFrame:
    clients = pd.read_csv(
        clients_path,
        parse_dates=["first_issue_date", "first_redeem_date"],
    )

    # Preserve the original age field but add a bounded version for modeling and
    # an explicit anomaly flag so bad source values are still auditable.
    clients["age_flagged"] = ((clients["age"] < 0) | (clients["age"] > 100)).astype(int)
    clients["age_clean"] = clients["age"].clip(lower=0, upper=100)

    # Expand the anonymized gender code into a model-friendly indicator set.
    clients["gender_M"] = (clients["gender"] == "M").astype(int)
    clients["gender_F"] = (clients["gender"] == "F").astype(int)
    clients["gender_U"] = (clients["gender"] == "U").astype(int)
    clients["gender_known"] = (clients["gender"] != "U").astype(int)

    # Redemption timing is structurally missing for customers who never redeemed,
    # so we pair every filled value with a missingness indicator.
    clients["ever_redeemed"] = clients["first_redeem_date"].notna().astype(int)
    clients["days_to_redeem"] = (
        clients["first_redeem_date"] - clients["first_issue_date"]
    ).dt.days
    clients["days_to_redeem_missing"] = clients["days_to_redeem"].isna().astype(int)
    clients["days_to_redeem"] = clients["days_to_redeem"].fillna(-1)

    clients["issue_age_days"] = (campaign_date - clients["first_issue_date"]).dt.days
    clients["redeem_recency_days"] = (
        campaign_date - clients["first_redeem_date"]
    ).dt.days
    clients["redeem_recency_missing"] = clients["redeem_recency_days"].isna().astype(int)
    clients["redeem_recency_days"] = clients["redeem_recency_days"].fillna(-1)
```

Listing 1: Selected client-level cleaning and redemption-feature logic from `phase2_features.py`

This block performs four tasks. First, it preserves the original age field while creating a bounded version (`age_clean`) and an explicit anomaly flag (`age_flagged`). Second, it expands the gender code into indicator variables that are directly consumable by the models. Third, it derives customer redemption timing variables, including whether the customer has ever redeemed at all. Fourth, it pairs structurally missing redemption values with missingness indicators before sentinel filling. In methodological terms, this excerpt operationalizes the Phase I conclusion that anomalous ages and redemption-related missingness should be retained as signals rather than silently discarded.

A.2 Product Behavior Aggregation

Listing 2 shows the essential part of the DuckDB aggregation used to convert line-item transaction history into client-level product behavior features. The full query is longer than necessary

for the report, so only the defining portion is shown.

```
    # Build one spend accumulator per observed hashed top-level category so the
    # final client-level matrix captures category mix without assuming semantics.
    category_spend_cols = ",\n".join(
        [
            f"    sum(case when level_1 = '{category}' then purchase_sum else 0 end) "
            f"as level1_spend_{category}"
            for category in level1_categories
        ]
    )

    conn = _connect()
    try:
        # Push the wide product-behavior aggregation into DuckDB so the largest
        # table never has to be materialized in pandas at full granularity.
        query = f"""
        with products_clean as (
          select
            product_id,
            level_1,
            brand_id,
            coalesce(is_alcohol, 0) as is_alcohol,
            coalesce(is_own_trademark, 0) as is_own_trademark,
            case
              when netto is null then null
              else least(netto, {netto_p99})
            end as netto_clean
          from read_csv(?, header=true, columns={PRODUCT_COLUMNS_SQL})
        ),
        joined as (
          select
            purchases.client_id,
            coalesce(purchases.purchase_sum, 0) as purchase_sum,
            coalesce(purchases.product_quantity, 0) as product_quantity,
            products_clean.level_1,
            products_clean.brand_id,
            products_clean.is_alcohol,
            products_clean.is_own_trademark,
            products_clean.netto_clean
          from read_csv(?, header=true, columns={PURCHASE_COLUMNS_SQL}) purchases
          left join products_clean
          using (product_id)
        ),
        client_rollup as (
          select
            client_id,
            sum(purchase_sum) as product_join_total_spend,
            sum(case when is_alcohol = 1 then purchase_sum else 0 end) as alcohol_spend,
            sum(case when is_own_trademark = 1 then purchase_sum else 0 end) as own_trademark_spend,
            sum(case when level_1 is null then purchase_sum else 0 end) as unknown_level1_spend,
            count(distinct brand_id) as n_brands,
            sum(coalesce(netto_clean, 0) * product_quantity) as netto_total,
            sum(case when netto_clean is null then product_quantity else 0 end) as missing_netto_quantity,
{category_spend_cols}
```

Listing 2: Selected DuckDB aggregation logic for product-behavior features from `phase2_features.py`

This excerpt achieves two things. The first is computational: it pushes the heaviest aggregation work into DuckDB so that the 4.4 GB purchases file does not need to be materialized in pandas at full granularity. The second is analytical: it defines the category spend accumulators, alcohol and own-trademark spend, netto-based measures, and brand concentration inputs that later become normalized client-level behavior signals. This is precisely how the implementation converts anonymized product metadata into usable uplift-model covariates without overstating semantic interpretability.

A.3 Deterministic Missing-Value Handling

Listing 3 shows the missing-value strategy used to ensure a complete numeric feature matrix for all models.

```python
def _fill_missing_with_indicators(
    train: pd.DataFrame,
    test: pd.DataFrame,
    id_columns: set[str],
) -> tuple[pd.DataFrame, pd.DataFrame, list[str]]:
    # All downstream models expect a fully numeric matrix. Missingness indicators
    # preserve information while deterministic sentinel filling removes NaNs.
    combined = pd.concat(
        [
            train.assign(_dataset="train"),
            test.assign(_dataset="test"),
        ],
        ignore_index=True,
        sort=False,
    )

    feature_columns = [
        column
        for column in combined.columns
        if column not in id_columns and column != "_dataset"
    ]

    for column in feature_columns:
        if combined[column].isna().any():
            indicator_col = f"{column}_missing"
            combined[indicator_col] = combined[column].isna().astype(int)

            observed = combined[column].dropna()
            if observed.empty:
                fill_value = -1.0
            elif (observed >= 0).all():
                # Use a shared negative sentinel for naturally non-negative fields.
                fill_value = -1.0
            else:
                fill_value = float(observed.min()) - 1.0

            combined[column] = combined[column].fillna(fill_value)
        combined[column] = pd.to_numeric(combined[column], errors="raise")
```

Listing 3: Selected deterministic missing-value handling logic from `phase2_features.py`

This block is methodologically important because it enforces a consistent contract across all model families. Whenever a feature contains missing values, the code first creates a dedicated missingness indicator and then fills the original field using a deterministic sentinel. Non-negative variables receive a shared negative sentinel, whereas variables with valid negative support receive a value below the observed minimum. The result is that no model consumes unresolved `NaN` values, while the fact of missingness remains available as an explicit predictor.

A.4 Linear Uplift Scoring Logic

Listing 4 presents the core scoring logic for the two logistic uplift baselines.

```python
        # Fit the treated and control response surfaces independently, then
        # compare their predicted probabilities at scoring time.
        self.treatment_model.fit(X[treatment == 1], y[treatment == 1])
        self.control_model.fit(X[treatment == 0], y[treatment == 0])
        return self

    def predict_uplift(self, X: np.ndarray) -> np.ndarray:
        treated_scores = self.treatment_model.predict_proba(X)[:, 1]
        control_scores = self.control_model.predict_proba(X)[:, 1]
        return treated_scores - control_scores


class LoInteractionLogisticUplift:
    def __init__(self, C: float, random_state: int, max_iter: int = 1000) ->
        None:
        self.C = C
        self.random_state = random_state
        self.max_iter = max_iter
        self.model = Pipeline(
            [
                ("scale", StandardScaler()),
                (
                    "logit",
                    LogisticRegression(
                        C=self.C,
                        max_iter=self.max_iter,
                        solver="lbfgs",
                        random_state=self.random_state,
                    ),
                ),
            ]
        )

    @staticmethod
    def _design_matrix(X: np.ndarray, treatment: np.ndarray) -> np.ndarray:
        # Lo-style uplift uses one shared linear model with explicit treatment
        # interactions rather than two separate conditional models.
        treatment = treatment.reshape(-1, 1).astype(float)
        return np.hstack([X, treatment, X * treatment])

    def fit(self, X: np.ndarray, treatment: np.ndarray, y: np.ndarray) -> "
        LoInteractionLogisticUplift":
        self.model.fit(self._design_matrix(X, treatment), y)
        return self

    def predict_counterfactual(self, X: np.ndarray, treatment_value: int) -> np
        .ndarray:
        treatment = np.full(X.shape[0], treatment_value, dtype=int)
        return self.model.predict_proba(self._design_matrix(X, treatment))[:,
            1]

```

```python
    def predict_uplift(self, X: np.ndarray) -> np.ndarray:
        return self.predict_counterfactual(X, 1) - self.predict_counterfactual(X, 0)
```

Listing 4: Selected scoring logic for the two-model and interaction-based logistic uplift estimators from `phase2_models.py`

This excerpt illustrates the two principal linear uplift formulations used in the project. In the two-model approach, separate treated and control response surfaces are estimated and then differenced at prediction time. In the Lo-style interaction approach, one shared model is fit on a treatment-augmented design matrix, and uplift is recovered as the difference between two counterfactual predictions for the same customer. These two formulations are conceptually distinct but mathematically comparable, which makes them suitable benchmark uplift models.

A.5 Benchmark and Non-Linear Uplift Wrappers

Listing 5 shows the essential logic behind the CCP benchmark and the non-linear uplift forest.

```python
        # Estimate the organic purchase propensity from untreated behavior only.
        control_mask = treatment == 0
        self.model.fit(X[control_mask], y[control_mask])
        return self

    def predict_score(self, X: np.ndarray) -> np.ndarray:
        return self.model.predict_proba(X)[:, 1]


class CausalMLUpliftRandomForest:
    def __init__(self, random_state: int, **params: Any) -> None:
        self.random_state = random_state
        self.params = params
        self.model = UpliftRandomForestClassifier(
            control_name="control",
            random_state=self.random_state,
            n_jobs=-1,
            evaluationFunction="KL",
            **self.params,
        )

    @staticmethod
    def _encode_treatment(treatment: np.ndarray) -> np.ndarray:
        # causalml expects string labels for the control/treatment arms.
        return np.where(treatment == 1, "treatment", "control")

    def fit(self, X: np.ndarray, treatment: np.ndarray, y: np.ndarray) -> "CausalMLUpliftRandomForest":
        self.model.fit(
            X.astype(np.float32),
            self._encode_treatment(treatment),
            y.astype(int),
        )
        return self

    def predict_uplift(self, X: np.ndarray) -> np.ndarray:
        return self.model.predict(X.astype(np.float32)).reshape(-1)
```

Listing 5: Selected benchmark and non-linear uplift model logic from `phase2_models.py`

The first half of the excerpt shows that the CCP benchmark is trained only on control observations, thereby estimating organic purchase propensity rather than treatment effect. The second half shows the uplift random forest wrapper, including the treatment-label encoding

required by the `causalml` API. Together, these two fragments illustrate the central contrast of the report: a conventional non-causal benchmark versus a dedicated non-linear causal-ranking model.

A.6 End-to-End Orchestration

Listing 6 shows how the pipeline rebuilds features, performs the outer validation split, and trains the four planned models.

```
def main() -> None:
    args = parse_args()
    args.output_dir.mkdir(parents=True, exist_ok=True)

    # Rebuild from raw CSVs unless the caller explicitly opts into cached
        features.
    bundle = build_phase2_feature_bundle(
        data_dir=args.data_dir,
        output_dir=args.output_dir,
        campaign_date=CAMPAIGN_DATE,
        reuse_cache=args.reuse_features,
    )
    feature_checks = validate_feature_bundle(bundle)

    avg_basket_value = (
        args.avg_basket_value
        if args.avg_basket_value is not None
        else bundle.metadata["avg_basket_value_empirical"]
    )

    outer_train, outer_valid = split_train_validation(
        bundle.train,
        seed=args.random_seed,
        validation_size=0.20,
    )

    model_results = [
        train_ccp_xgboost(
            outer_train,
            outer_valid,
            bundle.test,
            bundle.feature_columns,
            args.random_seed,
            avg_basket_value,
            args.sms_cost,
        ),
        train_two_model_logistic(
            outer_train,
            outer_valid,
            bundle.test,
            bundle.feature_columns,
            args.random_seed,
            avg_basket_value,
            args.sms_cost,
        ),
        train_lo_logistic(
            outer_train,
            outer_valid,
            bundle.test,
            bundle.feature_columns,
            args.random_seed,
            avg_basket_value,
            args.sms_cost,
        ),
```

```
        train_uplift_random_forest(
            outer_train,
            outer_valid,
            bundle.test,
            bundle.feature_columns,
            args.random_seed,
            avg_basket_value,
            args.sms_cost,
        ),
    ]
```

Listing 6: Selected training-stage orchestration from `phase2_pipeline.py`

This excerpt demonstrates that the entire Phase II workflow is driven from raw inputs and a small CLI configuration, rather than from manually prepared notebook artifacts. It also shows that the model comparison is operationally fair: the same feature bundle, validation split, and business assumptions are passed into each estimator.

Listing 7 shows how the validation scores are converted into ranked outputs, evaluated, and persisted as reusable artifacts.

```
    for result in model_results:
        score_column = result.model_name.lower().replace(" ", "_").replace("-",
            "").replace("__", "_") + "_score"
        rank_column = score_column.replace("_score", "_rank")

        scored_validation[score_column] = result.validation_scores
        scored_validation[rank_column] = (
            scored_validation[score_column].rank(ascending=False, method="first
                ").astype(int)
        )

        scored_test[score_column] = result.test_scores
        scored_test[rank_column] = scored_test[score_column].rank(ascending=
            False, method="first").astype(int)

        evaluation = evaluate_ranking(
            y_true=outer_valid["target"].to_numpy(dtype=int),
            treatment=outer_valid["treatment_flg"].to_numpy(dtype=int),
            scores=result.validation_scores,
            avg_basket_value=avg_basket_value,
            sms_cost=args.sms_cost,
        )

        summary_values = dict(evaluation.summary)
        if result.score_kind != "propensity":
            summary_values["predictive_mp"] = np.nan
            summary_values["predictive_optimal_target_share"] = np.nan

        metric_rows.append(
            {
                "model_name": result.model_name,
                "score_kind": result.score_kind,
                "tuning_metric": result.tuning_metric,
                **summary_values,
            }
        )

        qini_frames.append(evaluation.qini_curve.assign(model_name=result.
            model_name))
        curve_frames.append(evaluation.grid_curve.assign(model_name=result.
            model_name))

        params_payload["models"][result.model_name] = {
```

```
            "score_kind": result.score_kind,
            "best_params": _to_builtin(result.best_params),
            "tuning_metric": float(result.tuning_metric),
        }

    summary_metrics = pd.DataFrame(metric_rows).sort_values("mpu", ascending=
        False).reset_index(drop=True)
    qini_curves = pd.concat(qini_frames, ignore_index=True)
    model_curves = pd.concat(curve_frames, ignore_index=True)

    sensitivity_df = pd.DataFrame()
    if not args.skip_sensitivity_sweep:
        sensitivity_df = _build_sensitivity_table(
            model_curves,
            default_basket_value=avg_basket_value,
            default_sms_cost=args.sms_cost,
        )

    scored_validation_path = args.output_dir / "phase2_scored_validation.csv"
    scored_test_path = args.output_dir / "phase2_scored_test.csv"
    summary_metrics_path = args.output_dir / "phase2_summary_metrics.csv"
    qini_curve_path = args.output_dir / "phase2_qini_curves.csv"
    liftup_curve_path = args.output_dir / "phase2_liftup_curves.csv"
    sensitivity_path = args.output_dir / "phase2_profit_sensitivity.csv"
    params_path = args.output_dir / "phase2_model_params.json"

    scored_validation.to_csv(scored_validation_path, index=False)
    scored_test.to_csv(scored_test_path, index=False)
    summary_metrics.to_csv(summary_metrics_path, index=False)
    qini_curves.to_csv(qini_curve_path, index=False)
    model_curves.to_csv(liftup_curve_path, index=False)
    if not sensitivity_df.empty:
        sensitivity_df.to_csv(sensitivity_path, index=False)
    params_path.write_text(json.dumps(_to_builtin(params_payload), indent=2),
        encoding="utf-8")
```

Listing 7: Selected evaluation and artifact-writing logic from `phase2_pipeline.py`

This final excerpt shows how the pipeline turns model outputs into submission-relevant deliverables. Each model produces validation scores, test scores, ranking columns, summary metrics, curve tables, and parameter logs. The evaluation object is therefore not merely printed to screen; it is written into structured artifacts that support auditability, figure generation, and downstream targeting analysis.

B Reproduction Commands

Execute the following commands from the project directory.

```
python -m pip install -r .\requirements.txt
python .\verify_phase1_findings.py
python .\phase2_pipeline.py --reuse-features
python .\verify_phase2_outputs.py
```

Listing 8: Commands used to reproduce the work

References

[1] Devriendt, F., Guns, T., and Verbeke, W. (2021). Learning to rank for uplift modeling. In the value-driven uplift modeling literature adapted here for campaign targeting.

[2] Lo, V. S. Y. (2002). The True Lift Model. A treatment-interaction approach to uplift-style modeling.